Chapter 18
Artificial Intelligence and Optimization

Abstract Artificial Intelligence (AI), and automated problem solving have been the epitome of computer science for many decades. While some of the lofty goals of completely automated reasoning have not come to pass, AI has nevertheless cemented its place as one of a number of useful techniques to bear on a hard problem. In this chapter we discuss tried and proven open-source tools and libraries which implement AI concepts. The tools include CLIPS (expert system) and ACL2 (theorem prover). The libraries include simulated annealing, genetic algorithm, machine learning using SVM and general backtracking.

Contents

18.1 Introduction to AI Problems **391**
18.2 CLIPS: C Language Integrated Production System **392**
18.3 ACL2: automatic theorem proving **394**
18.4 GAUL : Genetic Algorithms Utility Library **397**
18.5 Representing floor-plans by $k-tuples$ **398**
18.6 ASA : Adaptive Simulated Annealing Library **400**
18.7 Artificial Neural Networks : FANN **402**
18.8 LIBSVM : Support Vector Machines **405**
18.9 Conclusion **408**

In this chapter we discuss open-source tools and libraries for performing artificial intelligence tasks, and problem solving.

18.1 Introduction to AI Problems

Artificial Intelligence is defined as the ability of computer systems to solve ill-defined problems using methods which are inspired by the human brain. Many techniques in AI, nevertheless, are derived from algorithms in graph theory and discrete mathematics (search, backtracking), thermodynamics (simulated annealing), biology (genetic algorithms and ant-colony optimization, neural networks). During the 1950s to 1980s, AI was thought to be a panacea, with which to solve all the complex

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_18, © Springer Science+Business Media, LLC 2011

problems of the day. But due to a confluence of factors, including lack of compute power and memory, over promised visions and other non-computer related factors, use of AI declined (this was termed the AI winter).

In the recent decade, AI has made a resurgence; even though researchers are still loath to actively claim a method as being AI. They refer to the method as machine learning, or guided optimization (to name some examples). Artificial Intelligence based computer programs are also find use in robotics where the persona of a human has to be adopted by the robot. In this chapter, however, we stick to *traditional* AI problems and present some of the advances which have been made. We present open-source solutions to many of the problems including developing an expert system, machine learning, general unconstrained optimization and the use of neural networks. Complete applications which solve an AI problem such as automatic theorem proving are also described.

18.2 CLIPS: C Language Integrated Production System

CLIPS is an expert system software, its name is an acronym for C Language Integrated Production System. On GNU/Linux system CLIPS can be started on the command line as:

```
$ ./source/clips/clips
         CLIPS (V6.0 05/12/93)
CLIPS>
```

Some useful functions with CLIPS are:

1. `(exit)`: quits CLIPS and returns back to the operating-system,
2. `(clear)`: removes all rules and facts from memory; equivalent to shutting down CLIPS and restarting,
3. `(reset)`: removes only facts (not rules) from memory,
4. `(run)`: starts executing the expert system.

Expert systems are comprised of *facts* about the universe in which they are expected to be experts, and *rules* which govern the relationships between the facts. CLIPS internally maintains a list of facts and rules. A fact can be *asserted* to CLIPS using the `(assert)` function. A fact is simply a piece of information deemed to be true:

```
CLIPS> (assert (direction east))
<Fact-0>
CLIPS> (assert (direction west))
<Fact-1>
CLIPS> (assert (direction north))
<Fact-2>
CLIPS> (assert (direction south))
<Fact-3>
```

Like facts in real life, sometimes they turn out to be false, in CLIPS a fact can be *retracted* as:

```
CLIPS> (assert (direction market))
<Fact-4>
CLIPS> (retract 4)
CLIPS> (facts)
f-0     (direction east)
f-1     (direction west)
f-2     (direction north)
f-3     (direction south)
For a total of 4 facts.
```

In the above example, we had added "market" as a direction (in some applications, a direction such as "head to the market" may make sense), but later we wished to remove this fact from CLIPS memory, and we used the `retract` function with the fact index to remove this fact. The function `(facts)` lists all facts known to CLIPS at this point of time. In addition to facts, an expert system also needs rules which combine facts. Consider the following example:

```
(defrule print-directions
   (direction ?dir)
=>
   (printout t ?dir " is a known direction" crlf))
(defrule street-name
   (direction street-name)
   =>
   (assert  (stname-direction name))
   (printout t "Street name has direction in it" crlf))
(run)
south is a known direction
north is a known direction
west is a known direction
east is a known direction
CLIPS>
```

CLIPS can also prompt the user for information, which is useful when writing an expert system to diagnose problems. The user can respond to the questions:

```
$ ~/EXPERT/source/clips/clips
         CLIPS (V6.0 05/12/93)
CLIPS> (defrule are-you-awake
(sleep ?)
=>
(printout t "Are you awake (yes or no)?")
(assert (awake (read))))
CLIPS> (assert (sleep sandeep))
<Fact-0>
```

```
CLIPS> (facts)
f-0     (sleep sandeep)
For a total of 1 fact.
CLIPS> (rules)
are-you-awake
For a total of 1 defrule.
CLIPS> (run)
Are you awake (yes or no)?Jack
CLIPS> (facts)
f-0     (sleep sandeep)
f-1     (awake Jack)
For a total of 2 facts.
```

Using CLIPS, it is simple to build up a chain of facts and rules and then the expert system can perform pattern matching and rule-chaining to arrive at the goal. Facts relevant to the domain can be stored in a database (or external file) and then loaded into CLIPS at run-time. Similarly, rules can be stored in a file and loaded into the memory. Expert systems in general, and CLIPS in particular, are examples of *data-driven* programming. It is instructive to see the program use data (facts) as the axis on which to guide the computation, as opposed to procedural programming where the functions manipulate the data.

18.3 ACL2: automatic theorem proving

ACL2 is an acronym for "A Computational Logic for Applicative Common Lisp." ACL2 is an interactive mechanical automated theorem prover which was designed for proving theorems modeling the behavior of hardware and software systems. In particular, ACL2 has been used to model and verify properties about the design of floating-point units of processors. ACL2 was written by Kaufmann and Moore as a successor to the Boyer-Moore theorem prover. ACL2 can be launched using a Common Lisp system which is loaded with the ACL2 generated core memory image as shown below:

```
$ ./saved_acl2
CMU Common Lisp 19f Fedora
    release 2.fc12 (19F), running on celex
With core: acl2-sources/saved_acl2.core
Dumped on:  on celex
Loaded subsystems:
    Python 1.1, target Intel x86
    CLOS based on Gerd's PCL 2008-11-12 16:36:41
Sat, 2010-06-12 03:10:36-07:00
ACL2 Version 3.6.  Level 1.
Cbd "/home/skoranne/acl2-sources/".
Distributed books directory "acl2-sources/books/".
Type :help for help.
```

```
Type (good-bye) to quit completely out of ACL2.

ACL2 !>
```

ACL2 is built on top of Common Lisp (see Section 1.6.8), thus all the features of Common Lisp are directly available inside ACL2. The "books" ACL2 is refering to in the above transcript refer to the collected knowledge that has been communicated to ACL2.

```
ACL2 !>(thm (equal (car (cons x y)) x ))

But we reduce the conjecture to T, by the
simple :rewrite rule CAR-CONS.

Q.E.D.

Summary
Form:  ( THM ...)
Rules: ((:REWRITE CAR-CONS))
Warnings:  None
Time:  0.00 seconds (prove: 0.00,
       print: 0.00, other: 0.00)

Proof succeeded.
```

In the above example we ask ACL2 to prove the identity which is latent in the definition of the Lisp function `car`. ACL2 uses the rewrite rule from its internal definitions to arrive at a proof. We can ask it to prove de'Morgan's law about Boolean algebra.

```
ACL2 !>(thm (implies (and a b) (not (or (not a) (not b)))))

But we reduce the conjecture to T, by case analysis.

Q.E.D.

Summary
Form:  ( THM ...)
Rules: NIL
Warnings:  None
Time:  0.01 seconds (prove: 0.00,
        print: 0.00, other: 0.01)

Proof succeeded.
```

The advantage of using Common Lisp as the underlying language to describe the theorem constituent is shown in the next example, where we define a new predicate `is-perm`:

```
(defun is-perm (x y)
"Is X a permutation of Y"
(if (consp x)
  (and (member (car x ) y )
  (is-perm (cdr x) (remove (car x) y)))
(not (consp y))))
```

This is a simple recursive function which checks where `(car x)` is a member of Y and if so, deletes that element and checks the remainder of the list. We can use the above defined predicate to check whether a sorting function is producing a permutation of the input. Consider the following definition of a `insert` function:

```
(defun insert (e x)
  (cond ((endp x) (cons e x))
    ((< e (car x)) (cons e x))
    (t (cons (car x) (insert e (cdr x))))))
```

We can test this function as:

```
ACL2 !>(defun insert (e x)
       (cond ((endp x) (cons e x))
             ((< e (car x)) (cons e x))
             (t (cons (car x)
                (insert e (cdr x))))))

The admission of INSERT is trivial, using the
relation O< (which is known to be well-founded on
the domain recognized by O-P) and the measure
(ACL2-COUNT X).  We observe that the type of
INSERT is described by the theorem
(CONSP (INSERT E X)).
We used primitive type reasoning.

Summary
Form:  ( DEFUN INSERT ...)
Rules: ((:FAKE-RUNE-FOR-TYPE-SET NIL))
Warnings:  None
Time:  0.00 seconds (prove: 0.00,
        print: 0.00, other: 0.00)
 INSERT
ACL2 !>(insert 3 '(1 2 5 6 7 8))
(1 2 3 5 6 7 8)
ACL2 !>
```

Using the `insert` function we can write a simple sorting function as:

```
(defun isort (x)
   (if (endp x) nil
     (insert (car x)
       (isort (cdr x)))))
```

And test it as:

```
Form:  ( DEFUN ISORT ...)
Rules: ((:TYPE-PRESCRIPTION INSERT))
Warnings:  None
Time:  0.00 seconds (prove: 0.00,
```

```
        print: 0.00, other: 0.00)
 ISORT
ACL2 !>(isort '(2 3 4 1 ))
(1 2 3 4)
```

To prove that indeed this sorting is producing an ordered output, as well as a permutation of the input we ask ACL2 to prove the latter automatically.

```
CL2 !>(defthm perm-isort
         (is-perm (isort x) x))
```

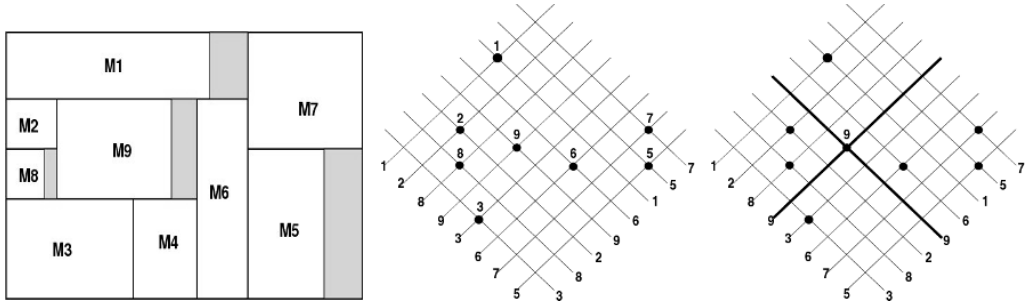
We try to assert the fact that a sorted output is basically a permutation of the input. This theorem is automatically proven by ACL2 (the output of the rules used is too large to include here). Above we have seen a simple example of using the Common Lisp sub-language within ACL2 to define functions and predicates over which ACL2 can perform automated reasoning. ACL2 can also be used deduce new theorems over knowledge fields, by systematic generation and refutation of new theorems. The theorems ACL2 can prove can then be added to its repertoire of knowledge for further sessions.

18.4 GAUL : Genetic Algorithms Utility Library

Genetic algorithms are an optimization method which use an iterative search technique based on evolution. Problem variables are modeled as *individuals*, and by combining traits of parents, the offsprings are expected to have a better objective function. The GAUL library implements several variations of genetic algorithms. The `entity` data-structure stores individuals, while `population` data-structure stores multiple entities. The population genesis per iteration is provided by the following function:

```
    *ga_genesis_integer( const int               population_size,
                         const int               num_chromo,
                         const int               len_chromo,
                         GAgeneration_hook       generation_hook,
 5                       GAiteration_hook        iteration_hook,
                         GAdata_destructor       data_destructor,
                         GAdata_ref_incrementor  data_ref_incrementor,
                         GAevaluate              evaluate,
                         GAseed                  seed,
10                       GAadapt                 adapt,
                         GAselect_one            select_one,
                         GAselect_two            select_two,
                         GAmutate                mutate,
                         GAcrossover             crossover,
15                       GAreplace               replace,
       vpointer    userdata );
```

We model a floor-planning problem using genetic-algorithms.



(a) Floor-plan (b) 45 degree oblique grid (c) Cone for block 9

Fig. 18.1 Sequence-pair representation

18.5 Representing floor-plans by $k-tuples$

We introduce the concept of $k-tuples$, which is an efficient and compact representation of floor-plans. The idea of $k-tuples$ is based upon the seminal work of Murata *et al.* in solving the rectangle placement problem using a notation called *sequence-pair*. Since $k-tuples$ are based on sequence-pair we give a brief introduction of sequence-pair notation below.

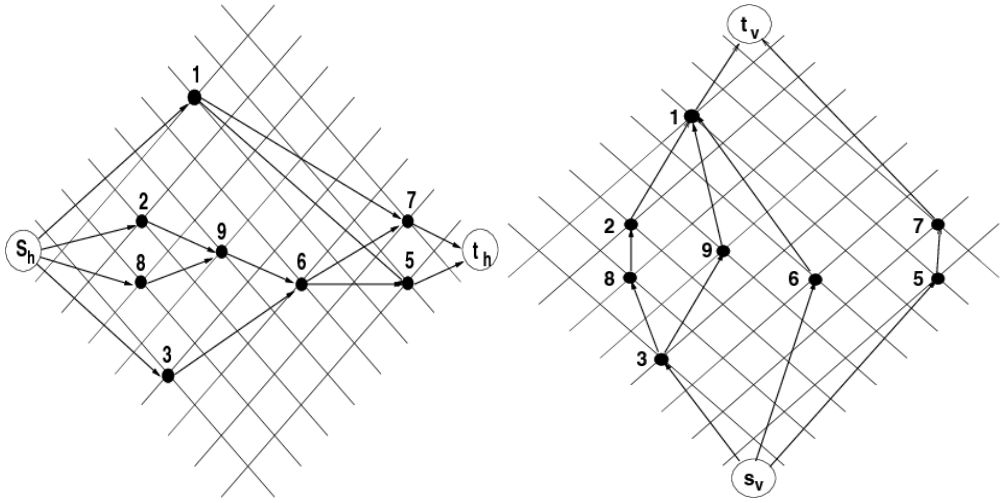
18.5.1 Sequence-pair Notation

A sequence-pair is a pair of sequences of n elements representing a list of n blocks. Given a block placement (like the one given in Figure 18.1(a)) Murata *et al.* describe a procedure for creating a linear order amongst the blocks using a 45-degree oblique grid notation as shown in Fig. 18.1(b). For every block, the plane is divided by the two crossing slope lines into four cones as shown in Figure 18.1 (c). Given a sequence-pair representation for a block placement (test schedule, equivalently), the horizontal topological relationship (the linear order amongst the block on the x axis) amongst the blocks can be represented as a horizontal constraint graph $G_h(V,E)$, which can be constructed as follows:

1. $V = \{s_h\} \cup \{t_h\} \cup \{v_i | i = 1,\ldots,n\}$, where v_i corresponds to a block, s_h is the source node representing the left boundary and t_h is the sink node, representing the right boundary;
2. $E = \{(s_h, v_i) | i = 1,\ldots,n\} \cup \{(v_i, t_h) | i = 1,\ldots,n\} \cup \{(v_i, v_j) |$ block v_i is to the left of block v_j $\}$.

The vertical constraint graph $G_v(V,E)$ can be similarly constructed. The corresponding constraint graphs $G_h(V,E)$ and $G_v(V,E)$ for the example schedule shown in Figure 18.1(a) are shown in Figure 18.2(a) and (b), respectively.

A small example of using GAUL to optimize a floor-planning problem is shown in Listing 18.1. We model each sequence-pair by a chromosome of 16-bit, and we define a cost function which returns true when the sequence-pair (as represented by



(a) Horizontal Constraint Graph (G_h) (b) Vertical Constraint Graph (G_v)

Fig. 18.2 Graph representation of floor-plans (transitive edges have been omitted.)

the chromosome) is a feasible solution. GAUL library then implements the actual genetic algorithm iteration.

```
// \file gaul_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of genetic algorithm optimization
#include <iostream>           // Program IO
#include <cassert>            // assertion
#include <cstdlib>            // exit
extern "C" {
#include <gaul.h>             // GAUL library
}
int main(int argc, char* argv[] ) {
  random_init(); // RNG init for good mix
  unsigned int num_blocks = 10; // there are 10 blocks in floorplan
  const unsigned int NUM_GEN = 100; // number of generations
  population *pop = NULL; // collection of solutions
  for(unsigned int i=0; i < NUM_GEN; ++i ) {
    if( pop ) ga_extinction( pop );
    random_seed( i );
    pop = ga_genesis_integer( num_blocks, // population size
            1,          // number chromosomes
            16,         // length chromosome
            NULL, NULL, NULL, NULL,
            cost_function,
            ga_seed_integer_random,
            NULL,
            ga_select_one_bestof2,
            ga_select_two_random,
            ga_mutate_integer_singlepoint_drift,
            ga_crossover_integer_singlepoints,
            NULL,
            NULL );


    ga_population_set_parameters(pop,
         GA_SCHEME_DARWIN,
         GA_ELITISM_PARENTS_SURVIVE,
         0.5,0.05,0.0 );
    ga_evolution( pop, NUM_GEN );
  }
  ga_extinction( pop );
```

```
40    std::cout << std::endl;
      return(0);
  }
```

Listing 18.1 Example of using GAUL for floorplanning

This use of genetic algorithm to solve floor-planning is one of the classic examples of genetic algorithm (since this was one of the early problems for which GA was used). It can be seen that genetic algorithms can offer an innovative solution, and with the advent of parallel processing, their use is expected to rise.

18.6 ASA : Adaptive Simulated Annealing Library

Simulated annealing (SA) is a stochastic optimization technique, which although slower than other analytical techniques, often provides better quality of solution. It can be used in general unconstrained combinatorial search applications where the objective function is not amenable to analytic implementation. However, there exist functions (in particular linear and quadratic forms) which can be optimized more efficiently using linear programming, or quadratic programming. Thus, the reader should always consider an analytic approach before deciding to use simulated annealing (as it can be many times slower).

To use SA with constrained problem, a technique called SUMS (scalable unconstrained minimization solver) can be used which converts the non-feasible region (lying outside the constraints to a very large cost region), thus allowing the annealer to avoid it. Simulated Annealing itself is inspired by thermodynamics, where crystalline forms were heated and allowed to cool slowly, thus forming perfectly aligned shapes. Similarly, SA is organized and implemented so that the search space is explored iteratively; in the starting the *temperature* is set to a high value. The objective function is evaluated at configurations, if the configuration has a lower cost of the objective function (for minimization) it is accepted, else, a random number generator is consulted, and a probability function is used to accept the configuration (even when the cost is more than the current configuration). This feature of accepting increasing cost configuration (albeit with a decreasing probability) allows SA to escape *local minima* and differentiates it from *greedy descent*.

The probability function is architected so that it depends on the temperature. When the temperature is high, more configurations are accepted. Iteratively, the temperature is reduced (*cooling*), and finally only configurations which reduce the cost are accepted. Choosing the initial starting temperature, number of trial configurations per temperature, iteration count, acceptance criteria, temperature schedule, random number generation and roll-back (reverting back to a known good configuration, if the cost becomes too high) are all known SA implementation criteria. There is significant theory involved in choosing the above correctly, but mostly they are experimentally chosen.

Adaptive Simulated Annealing (ASA) is a variation of SA where the cooling schedule and other factors listed above are automatically controlled by the anneal-

ing software. Once such library is the ASA library. To use the ASA library, the user has to define a cost function, acceptance criteria and configuration space exploration function. ASA library then automatically configures (adaptively), the cooling schedule and other metrics for efficient simulated annealing. The cost function is the called in the inner-loop of the SA algorithm, so care must be taken that implementation of the cost function is efficient. The user cost function and acceptance criteria can be defined as:

```
  /* user-defined */
  double USER_COST_FUNCTION (double *cost_parameters,
                             double *parameter_lower_bound,
                             double *parameter_upper_bound,
                             double *cost_tangents,
                             double *cost_curvature,
                             ALLOC_INT * parameter_dimension,
                             int *parameter_int_real,
                             int *cost_flag,
                             int *exit_code, USER_DEFINES * USER_OPTIONS);

// Acceptance criteria test function
  void user_acceptance_test (double current_cost,
                             double *parameter_lower_bound,
                             double *parameter_upper_bound,
                             ALLOC_INT * parameter_dimension,
                             const void *OPTIONS_TMP);

// Search space exploration, called generating function
  double user_generating_distrib (LONG_INT * seed,
                                  ALLOC_INT * parameter_dimension,
                                  ALLOC_INT index_v,
                                  double temperature_v,
                                  double init_param_temp_v,
                                  double temp_scale_params_v,
                                  double parameter_v,
                                  double parameter_range_v,
                                  double *last_saved_parameter,
                                  const void *OPTIONS_TMP);
```

To optimize the performance, the user can also specify a *re-anneal* cost, which is used by ASA to decide when to call annealing on a configuration space.

```
  int user_reanneal_cost (double *cost_best,
                          double *cost_last,
                          double *initial_cost_temperature,
                          double *current_cost_temperature,
                          const void *OPTIONS_TMP);
```

For self-optimization, the cost function maybe called recursively, the declaration of the recursive cost function is:

```
  double RECUR_USER_COST_FUNCTION (double *recur_cost_parameters,
                                   double *recur_parameter_lower_bound,
                                   double *recur_parameter_upper_bound,
                                   double *recur_cost_tangents,
                                   double *recur_cost_curvature,
                                   ALLOC_INT * recur_parameter_dimension,
                                   int *recur_parameter_int_real,
                                   int *recur_cost_flag,
                                   int *recur_exit_code,
                                   USER_DEFINES * RECUR_USER_OPTIONS);
```

ASA library has been used in many application where multi-dimensional search space optimization using simulated annealing was required, and is stated to be much faster than standard Boltzman annealing. Since a lot of the asa_usr code is used in the final application (only the cost function, and generating function templates in the code have to be modified), this library is not used as a conventional API library. In fact the code is modified and compiled to create the user application. We compiled and ran the code as:

```
gcc -O3 -o asa_test asa_usr.c asa_usr_cst.c asa.c -lm
```

where we had modified the templates in asa_usr.c and asa_usr_cst.c to reflect our application.

18.7 Artificial Neural Networks : FANN

Artificial Neural Networks (ANN) are biologically inspired graphs which exhibit *solution learning* behavior. By *training* an ANN on a set of problem-solution pairs, the network adapts its internal weights to seemingly become intelligent at solving new problem of the same nature. This description also contains the inherent limitation of ANNs, that the problem class has to be sufficiently narrow, or the learning time sufficiently large (alongwith memory to store the network) to solve a wider variety of problems. Even with this caveat, ANNs prove to be an useful tool in applications such as speech and hand writing recognition, where classical methods perform poorly.

ANNs have inputs, a computation network and a set of outputs. The computation network is defined using activation functions which define when a particular neuron should *fire* based on the strength of its inputs (and a function on them).

Implementing ANNs requires a good deal of good deal of knowledge on the workings of neural networks, however, the Fast ANN library already implements most of the commonly used networks. Using FANN in an application is straightforward as shown in Listing 18.2.

```
// \file fann_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using FANN
// Fast Artificial Neural Network library
#include <iostream>           // program IO
#include <cassert>            // assertion checking
#include <fann.h>             // FANN library

int main(int argc, char* argv [] ) {
  float connection_rate = 1.0;
  float learning_rate = 0.85;
  unsigned int number_layers = 3;
  unsigned int number_inputs = 16;
  unsigned int number_outputs = 4;
  unsigned int number_internals = 10;
  struct fann *nw =
    fann_create_standard( number_layers, number_inputs,
        number_internals, number_outputs );
```

```
20    assert( nw && "Unable to construct ANN" );
      fann_train_on_file( nw, "classify.dat", 1000,50,0.001 );
      fann_save( nw, "bucket_classify.net" );
      fann_destroy( nw );
      std::cout << std::endl;
25    return (0);
    }
```

Listing 18.2 Example of using FANN neural network library

In Listing 18.2 we show the creation and training of an artificial neural network (ANN) using the FANN library. In the code we create a network using the `fann_create_standard` which takes as input the number of layers of the network, number of inputs, internal layers and number of outputs. Consider a data grouping or classification application which classifies a given input set of 16 data values into one of four categories; thus number of inputs is 16 and number of outputs is 4 (binary). The network has to be trained on a sample set. In this example we generate a sample data file. The header of this file states that there are 5 patterns of 16 inputs and 4 outputs. Compiling the program in Listing 18.2 and running it we get:

```
$ cat classify.dat
2 16 4
0.1 0.4 0.6 0.3 0.8 0.1 0.1 0.0 0.2 0.1 0.4 0.6 0.3 0.8 0.1 0.1
1 0 0 0
0.7 0.64 0.66 0.3 0.58 0.2 0.1 0.0 0.2 0.2 0.4 0.6 0.3 0.8 0.1 0.1
0 0 0 1
0.1 0.4 0.6 0.3 0.8 0.1 0.1 0.0 0.2 0.1 0.4 0.6 0.3 0.8 0.1 0.1
0 1 0 0
0.1 0.14 0.6 0.3 0.8 0.1 0.1 1.0 1.0 0.1 0.4 0.6 0.3 0.8 0.1 0.1
0 0 1 0

$ ./fann_example
Max epochs     1000. Desired error: 0.0010000000.
Epochs            1. Current error: 0.9598405957. Bit fail 8.
Epochs           50. Current error: 0.2475839257. Bit fail 1.
Epochs           88. Current error: 0.0007690609. Bit fail 0.
```

The result of this training session is the file 'bucket_classify.net' which contains a representation of the ANN:

```
FANN_FLO_2.1
num_layers=3
learning_rate=0.700000
connection_rate=1.000000
network_type=0
learning_momentum=0.000000
training_algorithm=2
train_error_function=1
train_stop_function=0
cascade_output_change_fraction=0.010000
quickprop_decay=-0.000100
quickprop_mu=1.750000
rprop_increase_factor=1.200000
rprop_decrease_factor=0.500000
```

```
rprop_delta_min=0.000000
rprop_delta_max=50.000000
rprop_delta_zero=0.500000
cascade_output_stagnation_epochs=12
... // more properties
neurons (num_inputs, activation_function,
         activation_steepness)=(0, 0,
0.00000000000000000000e+00)
```

The network file contains the network property and the activation function for the neurons. Once the network has been created and stored in a file it can be used for future classification as shown in Listing 18.3.

```
// \file fann_use.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of FANN produced network for classification
#include <iostream>           // program IO
#include <cassert>            // assertion checking
#include <fstream>            // for file IO
#include <fann.h>             // FANN library

int main( int argc, char *argv [] ) {
  if( argc != 3 ) {
    std::cerr << "Usage : ./fann_use <nw> <data>...\n";
    exit(1);
  }
  struct fann *nw = fann_create_from_file( argv[1] );
  assert( nw && "Unable to create network from file" );
  float data[16];        // remember 16 inputs
  std::ifstream ifs( argv[2] );
  unsigned int count = 0;
  while( ifs ) {
    ifs >> data[count++];
  }
  float *classification; // remember 4 outputs
  classification = fann_run( nw, data );
  std::cout << "Classified data set = "
      << "\nData[0] probability = " << classification[0]
      << "\nData[1] probability = " << classification[1]
      << "\nData[2] probability = " << classification[2]
      << "\nData[3] probability = " << classification[3];

  std::cout << std::endl;
  return (0);
}
```

Listing 18.3 Using FANN produced ANN for classification

Compiling and running the program on a new sample data (of 16 inputs) we get:

```
$ ./fann_use bucket_classify.net sample.dat
Classified data set =
Data[0] probability = -0.001551
Data[1] probability = 1.85513
Data[2] probability = 3.20333e-32
Data[3] probability = 0.23337
```

Thus, we conclude the sample data can be classified in the second category. The above examples are deliberately simple, but FANN is used in this manner for complex classification tasks where network training is comprised of many millions of

trials on valid data. The simplicity of the API is well demonstrated using these small examples. Some of the other useful functions from FANN are given below in Table 18.1.

Table 18.1 FANN API Functions

Name
`fann * fann_create_standard(unsigned int num_layers, ...);`
`fann * fann_create_sparse(float connection_rate)`
`fann * fann_create_sparse_array(float connection_rate)`
`fann * fann_create_shortcut(unsigned int num_layers, ...);`
`fann * fann_create_shortcut_array(unsigned int num_layers)`
`void fann_destroy(fann *ann);`
`fann_type * fann_run(fann *ann, fann_type * input);`
`void fann_randomize_weights(fann *ann, fann_type min_weight)`
`void fann_init_weights(fann *ann, fann_train_data *train_data);`
`void fann_print_connections(fann *ann);`
`void fann_print_parameters(fann *ann);`
`unsigned int fann_get_num_input(fann *ann);`
`unsigned int fann_get_num_output(fann *ann);`
`unsigned int fann_get_total_neurons(fann *ann);`
`unsigned int fann_get_total_connections(fann *ann);`
`enum fann_nettype_enum fann_get_network_type(fann *ann);`
`float fann_get_connection_rate(fann *ann);`
`unsigned int fann_get_num_layers(fann *ann);`
`void fann_get_layer_array(fann *ann, unsigned int *layers);`
`void fann_get_bias_array(fann *ann, unsigned int *bias);`
`void fann_get_connection_array(fann *ann)`
`void fann_set_weight_array(fann *ann`
`void fann_set_weight(fann *ann`
`void fann_set_user_data(fann *ann, void *user_data);`
`void * fann_get_user_data(fann *ann);`
`unsigned int fann_get_decimal_point(fann *ann);`
`unsigned int fann_get_multiplier(fann *ann);`

18.8 LIBSVM : Support Vector Machines

Support Vector Machines are a class of AI tools and techniques used for automatic data classification. LIBSVM is an integrated software for support vector classification (C-SVC), regression estimation, and distribution estimation. It implements an SMO-type algorithm and is available as an open-source library.

Like Artificial Neural Networks (ANN) (see Section 18.7) tasks involved in SVM classification can be divided into (i) training, and (ii) actual classification. The goal of the training data sets is to construct a model (based on the training data) which predicts the target value of the test data (given only the test data attributes). The mathematical principles behind SVM are based on the mapping of the training vectors into a higher (maybe infinite) dimensional space. SVM then calculates a linear separating hyperplane with the maximal margin in this higher dimensional space.

Due to the principle of dimensional mapping, SVM requires that each data instance be represented as a vector of real numbers. Hence, if the data consists of categorical attributes, these have to be converted into numeric data. A simple *one-hot* encoding can be used to encode attribute data, using Boolean 0 and 1 to represent exclusion and inclusion in the category set. For example, the pieces of a chess game can be represented by the tuple:

$$CP = rook, bishop, knight \qquad rook = 1,0,0 \qquad bishop = 0,1,0 \qquad knight = 0,0,1$$

18.8.1 SVM Tools

LIBSVM consists of a number of tools which are described below:

1. svm-train: The command-line usage of this tool is shown below:

```
$ ./svm-train
Usage: svm-train [options]
       training_set_file [model_file]
options:
-s svm_type : set type of SVM (default 0)
-t kernel_type : type kernel function
-d egree : degree in kernel function (default 3)
-g gamma : gamma in kernel function
-r coef0 : coef0 in kernel function (default 0)
-c cost : C of C-SVC, epsilon-SVR (default 1)
-n nu : nu of nu-SVC, (default 0.5)
-p epsilon : epsilon-SVR (default 0.1)
-m cachesize : cache size in MB (default 100)
-e epsilon : tolerance of termination
-h shrinking : use shrinking heuristics
-b probability_estimates :
   whether to train a SVC or SVR model
   for probability estimates, 0 or 1
-wi weight : C of class i to weight*C
-v n: n-fold cross validation mode
-q : quiet mode (no outputs)
```

2. svm-predict: The command-line usage for this tool is shown below:

```
$ ./svm-predict
Usage: svm-predict [options] test_file
       model_file output_file
```

```
options:
-b probability_estimates:
  whether to predict probability estimates,
  0 or 1 (default 0);
  for one-class SVM only 0 is supported
```

3. svm-scale: performs scaling of the data prior to classification. Scaling is an important pre-processing check which can improve the quality of the classification. The example usage of this program is shown below:

```
$ ./svm-scale
Usage: svm-scale [options] data_filename
options:
-l lower : x scaling lower limit (default -1)
-u upper : x scaling upper limit (default +1)
-y y_lower y_upper :
  y scaling limits (default: no y scaling)
-s save_filename :
  save scaling parameters to save_filename
-r restore_filename :
  restore scaling parameters from restore_filename
```

4. checkdata: a simple Python program which checks the training data for syntax correctness,

The usual method of classifying data with LIBSVM is to train the SVM model on pre-defined training data (using `svm_train`), and then perform actual classification using `svm_predict`. The format of the data (both training and classification) is shown below:

```
<label> <index1>:<value1> <index2>:<value2> ..
```

where each line denotes an instance and is ended by the newline character. For classification, `<label>` is an integer indicating the class label. The `<index>:<value>` are index, value where indices start from 1 and value is a real number. Consider the same classification data which we used for the artificial neural network (except the attributes have been reduced to 4):

```
+1 1:0.1 2:0.4 3:0.6 4:0.3
-1 1:0.7 2:0.64 3:0.66 4:0.3
+1 1:0.1 2:0.4 3:0.6 4:0.3
+1 1:0.1 2:0.14 3:0.6 4:0.3
```

The data file should not have blank trailing lines. The Python program `checkdata` can be used to check the data for syntax. We now train our SVM model on this data using:

```
$ ~/libsvm-2.91/svm-train data.dat
*
optimization finished, #iter = 2
nu = 0.500000
obj = -1.900054, rho = -0.900054
```

```
nSV = 2, nBSV = 2
Total nSV = 2
```

The produced SVM model is shown below:

```
svm_type c_svc
kernel_type rbf
gamma 0.25
nr_class 2
total_sv 2
rho -0.900054
label 1 -1
nr_sv 1 1
SV
1 1:0.1 2:0.4 3:0.6 4:0.3
-1 1:0.7 2:0.64 3:0.66 4:0.3
data.dat.model (END)
```

We can now use this model to perform data classification on real data as shown below:

```
$./cat test.dat
+1 1:0.3 2:0.4 3:0.6 4:0.31
-1 1:0.67 2:0.64 3:0.46 4:0.23
+1 1:0.1 2:0.4 3:0.61 4:0.33
+1 1:0.1 2:0.14 3:0.16 4:0.28
$./svm-predict -b 0 test.dat data.dat.model output
Accuracy = 75% (3/4) (classification)
```

It is also possible to use LIBSVM within an application by using the classification facility as an API. The model creation and training can be carried out, and the model loaded into another application which performs the classification.

18.9 Conclusion

Artificial Intelligence (AI) has been a promised goal of computer science research for more than half a century. Nevertheless, recent years has seen many of the tools and techniques which were invented during AI's inception, find use in diverse application domains. In this chapter we have presented the CLIPS expert system, ACL2 theorem prover and a number of API libraries which implement search and optimization techniques such as simulated annealing, genetic algorithms, support vector machines and artificial neural networks. Since AI is often used to solve optimization problems with poorly defined, or ill-conditioned constraints, it is important to know of current state-of-art in AI tools and techniques when faced with difficult optimization problems which may be amenable to AI techniques.

Part V
Scientific Visualization